Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ДОМАШНЕЕ ЗАДАНИЕ №1

«Реализация алгоритма поиска с опечатками»

ДИСЦИПЛИНА: «Программирование на основе классов и шаблонов»

Выполнил: студент гр. ИУ5-22Б _____ (Краснова А. А.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю. Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Москва, 2026 г.

Условия домашнего задания №1

- Домашнее задание реализуется в виде консольного приложения на языке программирования C#. Рекомендуется использовать операторы верхнего уровня.
- Пользователь вводит две строки, после чего программа выводит значение расстояния Левенштейна между этими строками.
- Указанные выше действия реализуются в виде вечного цикла. Для выхода из цикла в качестве первой строки вводится значение «exit».
- Для сдачи домашнего задания необходимо продемонстрировать работоспособность программы, ответить на вопросы и предоставить отчет, содержащий текст программы и результаты ее выполнения.

Основной код программы:

```
using System;
Console.WriteLine("Реализация алгоритма поиска с опечатками");
Console.WriteLine("Программа вычисления расстояния Левенштейна");
Console.WriteLine("Для выхода введите 'exit' в качестве первой строки\n");

while (true)
{
    Console.Write("Введите первую строку: ");
    string str1 = Console.ReadLine();

    if (str1?.ToLower() == "exit")
    {
        Console.WriteLine("Программа завершена.");
        break;
    }

    Console.Write("Введите вторую строку: ");
    string str2 = Console.ReadLine();

    // Вычисление расстояния Левенштейна
    int distance = LevenshteinDistance(str1, str2);

    Console.WriteLine($"Расстояние Левенштейна между '{str1}' и '{str2}': {distance}\n");
}

static int LevenshteinDistance(string s1, string s2)
{
    // Проверка на null
    if (s1 == null || s2 == null)
        return -1;

    int s1len = s1.Length;
    int s2len = s2.Length;

    if (s1len == 0 && s2len == 0)
        return 0;

    if (s1len == 0)
        return s2len;

    if (s2len == 0)
        return s1len;

    // Приведение строк к верхнему регистру для регистронезависимого сравнения
    string str1 = s1.ToUpper();
    string str2 = s2.ToUpper();

    int[,] matrix = new int[s1len + 1, s2len + 1];

    for (int i = 0; i <= s1len; i++)
        matrix[i, 0] = i;
```

```
    for (int j = 0; j <= s2len; j++)
        matrix[0, j] = j;

    for (int i = 1; i <= s1len; i++)
    {
        for (int j = 1; j <= s2len; j++)
        {
            int cost = (str1[i - 1] == str2[j - 1]) ? 0 : 1;
            int deletion = matrix[i - 1, j] + 1;
            int insertion = matrix[i, j - 1] + 1;
            int substitution = matrix[i - 1, j - 1] + cost;

            matrix[i, j] = Math.Min(Math.Min(deletion, insertion), substitution);
        }
    }
    return matrix[s1len, s2len];
}
```

Результат выполнения программы:

```
Консоль отладки Microsoft Visual Studio
Реализация алгоритма поиска с опечатками
Программа вычисления расстояния Левенштейна
Для выхода введите 'exit' в качестве первой строки

Введите первую строку: пример
Введите вторую строку: при12мер
Расстояние Левенштейна между 'пример' и 'при12мер': 2

Введите первую строку: пример
Введите вторую строку: 1при2мер3
Расстояние Левенштейна между 'пример' и '1при2мер3': 3

Введите первую строку: прИМер
Введите вторую строку: прМИер
Расстояние Левенштейна между 'прИМер' и 'прМИер': 2

Введите первую строку: ИВАНОВ
Введите вторую строку: БАННВО
Расстояние Левенштейна между 'ИВАНОВ' и 'БАННВО': 4

Введите первую строку: exit
Программа завершена.

C:\Users\Александра\source\repos\Homework1\Homework1\bin\Debug\net10.0\Homework1.exe (процесс 5372) завершил работу с ко
дом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Ав
томатически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Ответы на контрольные вопросы:

1. ***Что такое расстояние Левенштейна?***

Расстоянием Левенштейна называется минимальное количество действий, необходимых для преобразования одного слова в другое. Основные операции, учитываемые при вычислении расстояния Левенштейна:

D (delete) — удаление символа

I (insert) — добавление символа

R (replace) — замена символа

M (match) — совпадение символов (не требует действий)

2. ***Чем отличается расстояние Левенштейна от расстояния Дамерау-Левенштейна?***

Расстояние Дамерау-Левенштейна — это усовершенствованная версия расстояния Левенштейна, которая дополнительно учитывает операцию транспозиции (перестановки двух соседних символов) как единичное действие.

При использовании расстояния Дамерау-Левенштейна за единичное расстояние принимаются следующие действия:

I (insert) — добавление символа

D (delete) — удаление символа

R (replace) — замена символа

T (transposition) — перестановка двух соседних символов

3. Объясните алгоритм Вагнера-Фишера

Алгоритм Вагнера-Фишера использует динамическое программирование для вычисления расстояния Левенштейна.

Как работает:

1) Создается матрица размером (M+1) × (N+1), где M и N — длины строк.
2) Заполнение матрицы:
    a. Первая строка: 0,1,2,...,N (вставки)
    b. Первый столбец: 0,1,2,...,M (удаления)
    c. Для каждой ячейки D[i,j] считаем минимум из трех вариантов:
    d. D[i-1, j] + 1 — удаление
    e. D[i, j-1] + 1 — вставка
    f. D[i-1, j-1] + cost — замена (0 если символы совпадают, 1 если разные)

Дополнительно для Дамерау-Левенштейна:

3) Проверяем возможность транспозиции (перестановки соседних символов)
4) Если символы можно переставить, учитываем D[i-2, j-2] + cost
5) Результат — значение в правом нижнем углу матрицы.

Сложность: O(M × N) по времени и памяти.